Track-A Action Anticipation: From Correctness Fixes to V-JEPA Clip SSL

What-Happens-Next-Modal Kaggle competition
Closed-world (from-scratch) track

May 15, 2026

Abstract

This report documents in detail every change made to the Track-A pipeline of the *Something-Something V2 action anticipation* project. The work proceeded in four phases. **Phase 1** closes a number of correctness and accounting gaps that were silently inflating validation scores and hurting generalisation: the validation split now points at the official held-out folder, the ResNet-50 backbone is initialised with the Kaiming He recipe used by all modern from-scratch ResNet recipes, mixed-precision training (AMP) is wired in, optimizer/scheduler/scaler state are persisted across restarts, and a new *trained-class index list* is recorded in each checkpoint so that the submission pipeline can mask never-trained logits at inference time. **Phase 2** adds four opt-in architectural and training enhancements: per-block Stochastic Depth (DropPath), an attention-pool head that strictly subsumes the legacy mean-consensus head, exponential-moving-average (EMA) model averaging with an automatic "best-of-(live, EMA)" checkpoint selection rule, and Test-Time Augmentation (TTA) with an automatic left-right class-pair remap at inference time. **Phase 3** replaces an earlier DINO-on-still-frames SSL prototype — which gave inconsistent gains because it was structurally incompatible with the TSM trunk — by a V-JEPA-style clip-level mask-denoising pretraining that operates on the full T-frame clip and *exercises* the Temporal Shift modules during pretraining. A non-strict `model.init_from` hook in the supervised trainer warm- starts the ResNet-50 trunk from the resulting SSL checkpoint. **Phase 4** attacks the remaining failure modes of small-data training: a class-balanced sampler and class-balanced loss [9] for the strongly skewed class distribution (162 to 3170 samples, a 20× ratio), a motion-aware time-reversal augmentation that pairs reversible verb classes (*opening / closing*, *moving up / down*, *from left to right* ↔ *from right to left*) so reversed clips contribute valid training signal, multi-scale Test-Time Augmentation, and a post-hoc logit adjustment [10] at inference time. All changes default to off, preserving byte-for-byte parity with the legacy pipeline; the test suite grows from 120 to 161 tests, all passing.

Contents

1 Context

1.1 Competition rules

The Kaggle competition *Can a model predict the future?* provides short video clips from the Something-Something V2 dataset and asks participants to predict the action that will occur next, given only the beginning of each clip. Track A (*Closed World*) is the focus of this work: **models must be trained from scratch using only the provided data**; no external dataset and no pretrained weights are allowed. Submissions are evaluated by Top-1 classification accuracy (with Top-5 as a secondary metric) on a held-out test set whose public/private split is hidden from participants.

1.2 Starting point and observed problems

Before the work in this report, the supervised Track-A pipeline was a TSM-ResNet50 trained with Adam, label smoothing, RandAugment-T temporal augmentations, FrameMixUp, an 80/20 internal split of the training data for validation, and a cosine learning-rate schedule. Its best public Kaggle scores were:

Submission	Internal val Top-1	Public Top-1
Initial run (epoch 29)	0.4582	0.3207
Resumed run (epoch 60)	0.5666	0.3584

The roughly 21 percentage-point gap between internal validation and the public leaderboard pointed at a collection of issues, the most important of which were identified in a project review:

1. **Leaky validation split**. The 80/20 split was carved out of `data/train/` rather than using the official held-out `data/val/` folder. Validation samples were therefore drawn from the same recording sessions and demographics as training, producing optimistically biased numbers.
2. **Suboptimal initialisation**. The ResNet-50 backbone was initialised with Xavier weights, which is well-suited to networks with sigmoid/tanh activations but is documented to underperform Kaiming He initialisation on deep ReLU networks trained from scratch.
3. **No mixed-precision training**. Forward and backward passes ran in full FP32, leaving $\approx 1.5{-}2\times$ of GPU throughput on the table on Ampere or newer hardware.
4. **Optimizer / scheduler / scaler state lost on resume**. Only the model weights and the epoch counter were checkpointed, so restarting a long run forced the optimizer (Adam moments) and the cosine LR schedule to restart from scratch as well.
5. **Missing class 27**. The "50 classes" described in the competition page is in practice 33 unique class indices on disk with class 27 entirely absent. The submission pipeline did nothing to prevent the network from emitting this never-trained label, meaning $\approx 3\%$ of test predictions could land on it.
6. **No EMA, no TTA, no SSL pretraining.** The pipeline used none of the well-known accuracy-boosting techniques that are standard for video classification on small datasets.
7. **Strong class imbalance.** The 33 trained classes range from ≈ 162 to ≈ 3170 samples, a $20\times$ ratio. Neither the sampler nor the loss accounted for this skew, so the classifier was biased toward head classes and rare classes were systematically under-predicted.
8. **No motion-aware data augmentation.** The pipeline used RandAugment-T and FrameMixUp but no augmentation that explicitly exploited the temporal structure of clips; in particular, time-reversal — which doubles the available training signal for every direction-sensitive verb pair — was missing.

The remainder of this document describes the changes that addressed each of these issues, in the order they were introduced.

2 Phase 1: Foundations

Phase 1 comprises five low-risk, high-return changes. All of them were introduced behind explicit configuration switches so that historical runs can still be reproduced bit-for-bit by setting the new switches to their "off" defaults.

2.1 Official validation split

Logic. A train/val split that overlaps with the test distribution understates the generalisation gap. Switching to the official `data/val/` folder makes the validation score a faithful proxy for the public leaderboard score and therefore drives early stopping, model selection and hyperparameter choices in the right direction.

Implementation. A new boolean `dataset.use_official_val` (default `false`) was added to `configs/data/default.yaml`. When set to `true`, the trainer bypasses the internal 80/20 split and instead enumerates every video under `data/val/`.

```
use_official_val = bool(cfg.dataset.get("use_official_val", False))
if use_official_val:
    val_dir_for_val = Path(cfg.dataset.val_dir).resolve()
    val_samples = collect_video_samples(val_dir_for_val)
    train_samples = all_samples
    print(f"[data] use_official_val=true: train={len(train_samples)}, "
          f"val={len(val_samples)} (from {val_dir_for_val})")
else:
    train_samples, val_samples = split_train_val(
        all_samples, val_ratio=float(cfg.dataset.val_ratio),
        seed=int(cfg.dataset.seed))
```

2.2 Kaiming He initialisation

Logic. For a Conv→BN→ReLU stack trained from scratch, the variance of pre-activations grows or shrinks geometrically with depth unless the initial Conv weights are scaled with the Kaiming He recipe [1, Sec. 2.2]:

$$\mathrm{Var}(W_{i,j}) = \frac{2}{\mathrm{fan\_out}}.$$

Xavier initialisation, which uses $2/(\mathrm{fan\_in} + \mathrm{fan\_out})$, under-shoots this variance by roughly $2\times$ per layer for ReLU networks and is empirically a poor choice for from-scratch ResNet training.

Implementation. The `_init_weights_xavier` method of `AvancedResNet50TSM` was renamed to `_init_weights` and rewritten to walk every submodule:

```
def _init_weights(self) -> None:
    for module in self.modules():
        if isinstance(module, nn.Conv2d):
            nn.init.kaiming_normal_(module.weight,
                mode="fan_out", nonlinearity="relu")
            if module.bias is not None:
                nn.init.zeros_(module.bias)
        elif isinstance(module, nn.BatchNorm2d):
            if module.weight is not None:
                nn.init.ones_(module.weight)
            if module.bias is not None:
                nn.init.zeros_(module.bias)
    nn.init.normal_(self.classifier.weight, mean=0.0, std=0.01)
    if self.classifier.bias is not None:
        nn.init.zeros_(self.classifier.bias)
```

The classifier head is initialised with a small-Gaussian ($\mathcal{N}(0, 0.01^2)$) following the original ResNet and TSM recipes; BN affine parameters are set to $\gamma = 1, \beta = 0$.

2.3 Automatic Mixed Precision (AMP)

Logic. On Ampere, Hopper and newer GPUs, running the forward pass in FP16 (or BF16) and only the master copy of the optimizer state in FP32 yields 1.5–2× throughput at no measurable accuracy cost when paired with PyTorch's gradient scaler.

Implementation. A new boolean `training.amp` (default `false`) gates a `torch.amp.GradScaler` that is created in `train.py` and threaded through the engine helpers. `train_one_epoch` and `evaluate_epoch` both received two new keyword arguments, `scaler` and `amp_dtype`, and wrap their forward and backward passes in `torch.amp.autocast(device_type="cuda", dtype=amp_dtype)`. When `scaler` is `None` (the default), the code path is byte-for-byte identical to the legacy full-precision implementation.

```
amp_enabled = bool(cfg.training.get("amp", False)) \
              and device.type == "cuda"
scaler = torch.amp.GradScaler("cuda", enabled=True) \
         if amp_enabled else None

# ...inside train_one_epoch:
with torch.amp.autocast(device_type="cuda",
                        enabled=use_amp,
                        dtype=amp_dtype):
    logits = model(videos)
    loss = loss_fn(logits, target)
if use_amp:
    scaler.scale(loss).backward()
    scaler.step(optimizer)
    scaler.update()
else:
    loss.backward()
    optimizer.step()
```

2.4 Persistent optimizer / scheduler / scaler state

Logic. A long training run inevitably gets restarted (scheduled maintenance, a transient OOM, an accidental `Ctrl-C`). If only the model weights are persisted, the optimizer's running moments, the cosine LR schedule and the GradScaler all reset, which causes a visible accuracy dip for several epochs after each restart and makes "resume from epoch k" a much weaker recovery than "never interrupted".

Implementation. On every *best-checkpoint* save, the trainer now stores `optimizer.state_dict()`, `cosine_scheduler.state_dict()` (when present) and `scaler.state_dict()` (when AMP is on) inside the existing `extra` dict of the checkpoint payload. The schema version is unchanged. On resume, those state dicts are loaded inside try/except blocks so a checkpoint produced by an older code path still resumes cleanly (the cosine scheduler then falls back to its legacy fast-forward).

```
ckpt_extra: dict[str, Any] = {
    "val_top1": ckpt_stats.top1,
    "val_top5": ckpt_stats.top5,
    "val_loss": ckpt_stats.loss,
    "epoch": epoch + 1,
    "optimizer_state_dict": optimizer.state_dict(),
    "trained_class_indices": trained_class_indices,
    "checkpoint_kind": ckpt_kind,
}
if cosine_scheduler is not None:
    ckpt_extra["scheduler_state_dict"] = cosine_scheduler.state_dict()
if scaler is not None:
    ckpt_extra["scaler_state_dict"] = scaler.state_dict()
```

2.5 Trained-class indices and untrained-logit masking

Logic. The provided dataset has 33 unique class indices in $[0, 32]$ but class 27 has no training samples. The classifier head is nonetheless 33-wide (its weight tensor was initialised at random for class 27) and at inference time argmax can therefore land on class 27 by accident, producing systematically wrong predictions for every test clip on which the random class 27 logit happens to be the largest.

Implementation. The trainer derives the set of class indices that actually have at least one training sample and writes it into the checkpoint:

```
trained_class_indices: list[int] = sorted(
    {int(label) for _, label in train_samples})
```

The submission pipeline reads this list back and builds an additive mask that pushes every never-trained class to $-\infty$ before `argmax`:

```
def _build_untrained_mask(trained_class_indices, num_classes, device):
    if not trained_class_indices:
        return None
    trained = {int(i) for i in trained_class_indices
               if 0 <= int(i) < num_classes}
    if len(trained) >= num_classes:
        return None
    mask = torch.zeros(num_classes, dtype=torch.float32, device=device)
    untrained = [c for c in range(num_classes) if c not in trained]
    mask[untrained] = -math.inf
    return mask
```

Older checkpoints that pre-date Phase 1 do not carry this list; in that case the mask is `None` and the legacy unmasked argmax path is used, preserving backward compatibility.

3 Phase 2: Architectural and Training Enhancements

Phase 2 adds four orthogonal opt-in features that compose cleanly with Phase 1. None of them change the behaviour of the legacy code paths when left at their defaults; together, they are bundled in a new experiment config `configs/experiment/track_a_phase2.yaml` that also flips on AdamW with a stronger weight decay.

3.1 Stochastic Depth (DropPath)

Logic. Stochastic Depth [2] is a stochastic regularisation technique that, during training, drops the residual branch of each ResNet bottleneck block with probability p_ℓ and rescales the kept contributions by $1/(1-p_\ell)$ so that the expectation over the random mask matches the no-drop forward pass:

$$y_\ell \;=\; x_\ell \;+\; \frac{b_\ell}{1-p_\ell}\, F_\ell(x_\ell), \qquad b_\ell \sim \text{Bernoulli}(1-p_\ell).$$

At inference time, all blocks are kept ($b_\ell = 1$ deterministically) so no rescaling is needed. The standard recipe uses a linear schedule that drops deeper blocks more aggressively:

$$p_\ell \;=\; p_{\max}\,\frac{\ell-1}{L-1},$$

where L is the total number of bottleneck blocks. For ResNet-50 $L = 16$ and a typical $p_{\max}$ is in $[0.05, 0.2]$.

Implementation. A parameter-free `DropPath` module implements the per-sample bernoulli mask. A helper `_inject_drop_path` walks the four ResNet stages, attaches a `DropPath` instance to each block (with the linearly-scaled probability), and rebinds the block's `forward` to a patched implementation that calls `self.drop_path(out)` on the residual branch immediately before the residual addition:

```
def _patched_bottleneck_forward(self, x: torch.Tensor) -> torch.Tensor:
    identity = x
    out = self.relu(self.bn1(self.conv1(x)))
    out = self.relu(self.bn2(self.conv2(out)))
    out = self.bn3(self.conv3(out))
    if self.downsample is not None:
        identity = self.downsample(x)
    out = self.drop_path(out) # <-- new
    return self.relu(out + identity)
```

`DropPath` adds no parameters and no buffers, so the model's `state_dict` layout is unchanged regardless of `model.drop_path_rate`. This is asserted by a unit test that compares state-dict key sets with and without DropPath enabled.

3.2 Attention-pool head

Logic. The legacy temporal head averages the per-frame features:

$$z = \frac{1}{T}\sum_{t=1}^{T}\phi(x_t), \qquad \widehat{y} = Wz + b.$$

A 1-query multi-head attention pool is a strict generalisation: when the attention weights are uniform it reproduces the mean exactly, but it can also concentrate on the most informative frame(s). Concretely, we introduce a learnable query token $q \in \mathbb{R}^{1\times D}$ and compute

$$z \;=\; \mathrm{LayerNorm}(\mathrm{MultiHeadAttn}(q,\, K{=}V{=}\Phi))\,,$$

where $\Phi \in \mathbb{R}^{T\times D}$ is the stack of frame features. With $T = 4$ and $D = 2048$, this adds about $D + 4D^2 \approx 16{,}780{,}288$ parameters, which is small compared to the $\approx$ 25M parameter ResNet-50 trunk.

Implementation. A new `AttentionPool` module wraps a single `nn.MultiheadAttention` layer plus a final `nn.LayerNorm`; the query token is initialised with a truncated normal of std 0.02. The supervised model exposes a new `model.head` switch (`"mean"` or `"attn"`) and routes features accordingly:

```
sequence_features = frame_features.view(batch_size, num_frames, -1)
if self.attn_pool is not None:
    consensus_features = self.attn_pool(sequence_features)
else:
    consensus_features = sequence_features.mean(dim=1)
```

When `model.head` = `"mean"` (the default) the `attn_pool` attribute is `None` and no extra parameters are added.

3.3 Exponential Moving Average (EMA) of model weights

Logic. Polyak averaging [4] maintains an exponentially-weighted average of the model's parameters,

$$\theta_{\mathrm{EMA}}^{(t+1)} \;=\; \alpha\,\theta_{\mathrm{EMA}}^{(t)} \;+\; (1-\alpha)\,\theta^{(t+1)}, \qquad \alpha \in [0.99, 0.9999],$$

which empirically gives smoother loss curves and a small but consistent top-1 boost ($\approx 0.3{-}1.0$ points) on most modern image and video classifiers.

Implementation. We use PyTorch's `torch.optim.swa_utils.AveragedModel` with a custom averaging function and `use_buffers=True` so BatchNorm running statistics are also tracked:

```
def _ema_avg_fn(avg_param, model_param, _num_averaged):
    return ema_decay * avg_param + (1.0 - ema_decay) * model_param

ema_model = torch.optim.swa_utils.AveragedModel(
    model, avg_fn=_ema_avg_fn, use_buffers=True).to(device)
```

`train_one_epoch` accepts a new `ema_model` keyword and calls `ema_model.update_parameters(model)` after every optimizer step. At the end of each epoch the trainer evaluates *both* the live and the EMA model and saves the better of the two:

```
if ema_stats is not None and ema_stats.top1 > val_stats.top1:
    ckpt_stats, ckpt_module, ckpt_kind = ema_stats, ema_model, "ema"
else:
    ckpt_stats, ckpt_module, ckpt_kind = val_stats, model, "live"

# ...later, before save_checkpoint:
module_to_save = (ckpt_module.module
                  if isinstance(ckpt_module,
                                torch.optim.swa_utils.AveragedModel)
                  else ckpt_module)
```

This selection rule is what lets downstream consumers (`submit.py`, `evaluate.py`) stay completely oblivious to the EMA toggle: the saved `model_state_dict` is always loaded with `build_model(...).load_state_dic` regardless of which copy won. The string `"live"` or `"ema"` is recorded in `extra["checkpoint_kind"]` for traceability.

3.4 Test-Time Augmentation with auto class-pair remap

Logic. Horizontal flip is a free *evaluation-time* augmentation *provided* that the predicted class is invariant to the flip. The Something-Something V2 subset used in this competition contains a small number of direction-sensitive classes, in particular:

- class 018: "Pulling something from left to right"
- class 019: "Pulling something from right to left"

For these, a flipped clip should predict the *paired* class. A naive flip-and-average TTA actually hurts accuracy on these two classes because it averages two confident but *disagreeing* predictions.

Implementation. We average softmax probabilities across the two views (original and flipped) and apply a learned permutation $\pi : \{0, \ldots, K-1\} \to \{0, \ldots, K-1\}$ to the flipped softmax so that paired classes are mapped back into agreement. The permutation is *auto-derived from the class folder names*, making it robust to any future change in the class set:

```
LR = "from_left_to_right"
RL = "from_right_to_left"
for idx, stem in stem_by_idx.items():
    if LR in stem:
        mirror_stem = stem.replace(LR, RL)
        for other_idx, other_stem in stem_by_idx.items():
            if other_stem == mirror_stem:
                perm[idx] = other_idx
                perm[other_idx] = idx
                break
```

(The numeric prefix is stripped before matching so `018_..._left_to_right` pairs with `019_..._right_to_left` without a code-level hard-code.) The TTA loop in `submit.py` then becomes:

```
logits = _logits_for_batch(model, video_batch, untrained_mask)
probs_total = torch.softmax(logits, dim=1)
n_views = 1
if tta_flip:
    flipped = torch.flip(video_batch, dims=[-1])
    flipped_probs = torch.softmax(
        _logits_for_batch(model, flipped, untrained_mask), dim=1)
    if flip_perm is not None:
        flipped_probs = flipped_probs.index_select(dim=1, index=flip_perm)
    probs_total = probs_total + flipped_probs
    n_views += 1
predictions.extend(int(p) for p in
    (probs_total / n_views).argmax(dim=1).cpu().tolist())
```

The configuration is opt-in (`training.tta`, `training.tta_flip`, both `false` by default) and read at inference time only; training is unaffected.

3.5 AdamW and the bundled experiment configuration

The trainer already supported AdamW; Phase 2 simply makes it the recommended optimizer for the new experiment by setting `training.optimizer: adamw` and `training.weight_decay: 0.05` in `configs/experiment/track_a_phase2.yaml`. Decoupling the weight-decay term from the gradient (which is what AdamW does, unlike plain Adam) is the right thing to do whenever a non-zero weight decay is used; it is also the optimizer of choice for ConvNeXt, ViT and modern ResNet recipes.

The `track_a_phase2.yaml` bundle composes all of Phase 1 and Phase 2:

```
defaults:
  - override /model: avanced_resnet50_tsm
  - override /augment: randaugment_t

model:
  drop_path_rate: 0.1
  head: attn
  head_num_heads: 4

dataset:
  num_frames: 4
  use_official_val: true

training:
  optimizer: adamw
  weight_decay: 0.05
  amp: true
  ema_enabled: true
  ema_decay: 0.999
  tta: true
  tta_flip: true
  scheduler_cosine: true
  warmup_epochs: 10
  label_smoothing: 0.1
```

4 Phase 3: Self-Supervised Pretraining (V-JEPA)

The single highest-impact change available within Track A is to give the trunk something useful to do with the unlabeled clips before the supervised loss starts back-propagating. Track A permits self-supervised pretraining on the *provided* unlabeled images (the labels are simply not consumed and the trunk starts from random initialisation), so we pretrain on the union of the `train/`, `val/` and `test/` folders.

4.1 Why the DINO-on-still-frames prototype was deprecated

Our first SSL attempt was a DINO-v1 [3] pipeline trained on still frames sampled independently from the provided videos (file `src/smth2smth/shared/data/still_frames_dataset.py` and `src/smth2smth/shared/models/dino_ssl.py`). In practice it produced *inconsistent* downstream gains — sometimes a small improvement, sometimes a clear regression versus the random-init baseline. Two structural issues explain why:

1. **DINO is invariance-trained.** The objective is to assign the same prototype to two augmented views of the *same image*, which actively pressures the trunk to become invariant to the small per-pixel changes that distinguish adjacent video frames — exactly the signal an action-anticipation model needs to keep.

2. **The TSM modules are never exercised.** DINO operates on isolated frames, so the channel-shift modules wrapped around every bottleneck's `conv1` (which carry $\approx 12.5\%$ of the input channels temporally) see no temporal context during pretraining. Roughly one in eight conv1 input channels was therefore left at its Kaiming He initial value when supervised training started.

The DINO files remain in the repository as a historical record, but the recommended SSL pipeline is now the V-JEPA-style one described in the rest of this section.

4.2 V-JEPA-style clip-level pretraining

V-JEPA (Joint-Embedding Predictive Architecture for Video) [7, 8] replaces the "invariance to augmented views" objective by "predict the masked parts of a clip in feature space". Concretely, the student encodes a *masked* clip while the EMA-updated teacher encodes the unmasked clip; a small predictor maps the student's representation to the teacher's space, and an L1 loss is applied only at the masked positions. This is the right inductive bias for our problem: the student is forced to use temporal context to fill in masked features, which makes the TSM channel-shift modules immediately useful.

We adapt the recipe to our TSM-equipped ResNet-50 trunk with $T = 4$ frames. Because there are only 4 frames per clip, full V-JEPA 3D multi-block tube masking in patch-token space is not appropriate; we use *frame-level* masking instead, which mirrors the V-JEPA objective at the granularity that maps naturally onto our CNN trunk.

4.3 Clip dataset for SSL

A new file `src/smth2smth/shared/data/clip_ssl_dataset.py` exposes two utilities:

- `collect_all_video_dirs(roots)` walks the provided directories and returns every video folder it finds. It handles both layouts in use: class-bucketed (`train/<class>/<video>/<frame>.jpg`) and flat (`test/<video>/<frame>.jpg`). Duplicates across roots are removed.

- `ClipSSLDataset` yields one (T, C, H, W) clip per video. Frame indices are picked by the same `pick_frame_indices` helper used by supervised training so the SSL run sees exactly the same temporal distribution as the supervised trainer. The per-frame transform is the project's standard `build_transforms(is_training=True)`, which includes RandAugment-T spatial augmentations *without* ImageNet normalisation (Track A forbids pretrained weights).

On the question of V-JEPA's data augmentations. The V-JEPA paper combines random resized crop, horizontal flip, colour jitter, gray-scale and Gaussian blur in a specific stack. We *do not* reproduce that augmentation stack verbatim: we reuse the project's existing RandAugment-T pipeline so that the SSL trunk sees exactly the same input distribution as the downstream

supervised trainer. Reproducing V-JEPA's stack would put a domain shift between pretraining and fine-tuning, which is the opposite of what we want. The *augmentation* contribution of V-JEPA that we *do* reproduce is the masking itself: n_{masked} frames per clip are replaced by zeros so the student must reconstruct their features from the unmasked context through the TSM channels.

4.4 Model: trunk + predictor + EMA teacher

`src/smth2smth/shared/models/vjepa_ssl.py` contains three pieces.

Trunk. `VJepaTrunk` is the *same* TSM-wrapped ResNet-50 as the supervised model's `backbone`: it calls `_make_temporal_shift_resnet` with `n_segment` $= T$, so every bottleneck block's `conv1` is replaced by `nn.Sequential(TemporalShift, conv1)`. The state-dict layout therefore matches `AvancedResNet50TSM.backbone.state_dict()` byte-for-byte, which lets the supervised trainer pick up the SSL checkpoint with a single `load_state_dict(..., strict=False)`.

Predictor. A small per-frame MLP (`Linear` $\to$ `GELU` $\to$ `Linear`, hidden dim 2048) maps the student's per-frame features into the teacher's space. The predictor exists so the encoder is not biased toward an identity student-to-teacher mapping on masked positions (Bardes et al., Sec. 3).

Composite. `VJepaModel` composes the trunk and the predictor. Two copies are instantiated at training time — a student and a teacher — with identical architecture; the teacher's weights are an EMA of the student's and are never updated by gradient descent.

4.5 Frame-level masking and the V-JEPA L1 loss

For each clip we sample $n_{\text{masked}} \sim \text{Uniform}[1, T-1]$ and zero out the corresponding frames. The student trunk encodes the masked clip; the teacher trunk encodes the unmasked clip with no gradient. The masked-position L1 loss is

$$\mathcal{L} \;=\; \frac{1}{|\mathcal{M}| \cdot D} \sum_{(b,t)\in\mathcal{M}} \left\| \hat{f}_{b,t} - f^{\text{tch}}_{b,t} \right\|_1 ,$$

where $\hat{f}_{b,t} = \text{Predictor}(\text{Student}(\tilde{x})_{b,t})$ is the predicted feature for clip b at frame t, $f^{\text{tch}}_{b,t} = \text{Teacher}(x)_{b,t}$ is the teacher's target, $\mathcal{M}$ is the set of masked (b,t) positions and $D = 2048$. Masked positions are the only ones that contribute to the loss; unmasked frames see no learning signal, which keeps the gradient focused on *predicting what is missing*.

```python
def vjepa_feature_loss(predicted, teacher, mask):
    if not mask.any():
        return predicted.sum() * 0.0 # zero with valid grad_fn
    diff = (predicted - teacher).abs() # (B, T, D)
    sel = mask.to(diff.dtype).unsqueeze(-1) # (B, T, 1)
    weighted = diff * sel
    n_elems = mask.sum().clamp_min(1).to(diff.dtype) * diff.shape[-1]
    return weighted.sum() / n_elems
```

4.6 Teacher EMA momentum schedule

The teacher's parameters are updated by $\theta_t \leftarrow m\,\theta_t + (1-m)\,\theta_s$ after every optimizer step, with m following a cosine schedule from $m_0 = 0.996$ at the start to $m_1 = 1.0$ at the end of training.

4.7 Trunk-only checkpoint format

After every epoch we save a small (≈ 95 MiB) checkpoint that contains *only* the trunk parameters, with both the `trunk.` and the inner `backbone.` prefix stripped. The result is a flat dictionary keyed exactly like a standard `torchvision.models.resnet50` state-dict, except that the bottleneck `conv1` keys already carry the TSM-wrapping index (e.g. `layer1.0.conv1.1.weight` instead of `layer1.0.conv1.weight`):

```
trunk_state_dict = {}
for k, v in student.state_dict().items():
    if k.startswith("trunk.backbone."):
        trunk_state_dict[k.removeprefix("trunk.backbone.")] = v
torch.save({"trunk_state_dict": trunk_state_dict,
            "epoch": epoch + 1}, out_path)
```

4.8 The `model.init_from` hook in `train.py`

The supervised trainer received a single new opt-in:

```
init_from = cfg.model.get("init_from") if hasattr(cfg.model, "get") \
            else None
if init_from:
    payload = torch.load(Path(str(init_from)).resolve(),
                         map_location=device, weights_only=False)
    trunk_state = payload.get("trunk_state_dict")
    prefixed = _ssl_trunk_to_supervised_keys(trunk_state)
    missing, unexpected = model.load_state_dict(prefixed, strict=False)
```

The helper `_ssl_trunk_to_supervised_keys` handles both flavours of SSL checkpoint that the repository can produce:

- Legacy DINO trunks whose keys come from a plain `torchvision` ResNet-50 (e.g. `layer1.0.conv1.weight`). The helper renames them to the TSM-wrapped form (`layer1.0.conv1.1.weight`) and prepends `backbone.`.
- V-JEPA trunks whose keys are already TSM-wrapped (`layer1.0.conv1.1.weight`). The rename regex is a no-op and the helper only prepends `backbone.`.

```
block_conv1_re = re.compile(
    r"^(layer[1-4]\.\d+\.conv1)\.(weight|bias)$")
out: dict[str, torch.Tensor] = {}
for k, v in trunk_state.items():
    match = block_conv1_re.match(k)
    if match is not None:
        new_k = f"{match.group(1)}.1.{match.group(2)}"
    else:
        new_k = k
    out[f"backbone.{new_k}"] = v
return out
```

A V-JEPA trunk loads with **318 backbone tensors covered, zero unexpected keys, and only the classifier (and, when enabled, the attention pool) left at their fresh-init values** — the exact behaviour the test suite pins (`tests/pipelines/test_init_from_vjepa.py`).

5 Phase 4: Class Imbalance, Motion Augmentation, Better Inference

Phase 4 attacks the failure modes that remain once a good trunk is in place. Three of them stem directly from the small, imbalanced nature of the provided training set; the fourth is a

free accuracy boost at inference time. All four are opt-in via the configuration and are bundled together in `configs/experiment/track_a_vjepa_finetune.yaml`.

5.1 Class imbalance: balanced sampling *and* balanced loss

Logic. The training set is heavily skewed: class counts range from ≈ 162 to ≈ 3170 samples, a $20\times$ ratio. A vanilla cross-entropy loss with uniform sampling effectively *down-weights* rare classes twice over (they are seen less often, *and* each sample contributes equally to the loss), and the resulting classifier tends to predict the head classes for most test clips. We fix this on two independent axes.

Balanced sampler. We replace the uniform shuffle of the `DataLoader` by a `WeightedRandomSampler` whose per-sample weight is $w_i = 1/\sqrt{n_{c(i)}}$, where $c(i)$ is the class of sample i and n_c is the class count. The square root is a robust compromise between fully-balanced sampling (which heavily oversamples the 162-sample classes) and uniform sampling (which ignores them). The policy is selected by `training.class_balance_sampler` $\in$ {none, inverse, sqrt_inverse}.

Balanced loss. We additionally weight each class in the cross-entropy by Cui et al.'s effective-number weights [9]: for class c with n_c samples,

$$w_c \;\propto\; \frac{1-\beta}{1-\beta^{n_c}}, \qquad \beta \in (0,1),$$

which interpolates between uniform weighting ($\beta \to 0$) and inverse-frequency weighting ($\beta \to 1$). We use the canonical $\beta = 0.999$. Classes with zero samples (e.g. class 27, see Section 2.5) receive $w_c = 0$, so they contribute neither to the sampling nor to the loss. The same weight vector is also routed through the soft-target cross-entropy used by label smoothing and FrameMixUp so the augmentations remain class-balanced.

```python
# Per-class effective-number weights (Cui et al. 2019).
def _class_factors(n_per_class, policy, beta=0.999):
    if policy == "cb":
        effective = [(1.0 - beta**max(n, 1)) / (1.0 - beta)
                     for n in n_per_class]
        return [1.0 / e if e > 0 else 0.0 for e in effective]
    # ... 'inverse', 'sqrt_inverse' branches ...
```

The two axes are configured independently (`training.class_balance_sampler`, `training.class_balance_loss`, `training.class_balance_beta`) so the user can run any combination of {sampler off, sampler sqrt}$\times$ {loss off, loss CB} and compare. The new `class_balance.py` module is fully unit-tested (`tests/shared/utils/test`

5.2 Motion-aware time-reversal augmentation

Logic. For verbs whose semantics are intrinsically direction-sensitive, the time-reversed clip belongs to a *paired* class — the reverse of opening is closing, the reverse of pushing left-to-right is pushing right-to-left, etc. Time-reversal is therefore a label-preserving augmentation *only* when we also remap the label to the paired class. Without the remap, a reversed clip injects flat-out wrong supervision.

Implementation. A new module `src/smth2smth/shared/data/time_reversal.py` hosts a small hand-curated table of reversible class pairs:

> Opening $\leftrightarrow$ Closing, Moving up $\leftrightarrow$ Moving down, Pushing left-to-right $\leftrightarrow$ Pushing right-to-left, Tilting toward camera $\leftrightarrow$ Tilting away from camera, plus a symmetric set (e.g. *Holding something*) where the reverse is the same class.

`build_time_reversal_table` matches the pairs against the actual class folder names on disk and returns two tensors: `perm[c]` = the class index of the time-reversed clip (`perm[c] = -1` if c is not reversible), and `allow_mask[c] = True` iff class c is reversible. Both are threaded into `VideoFrameDataset` via three new keyword arguments:

```
class VideoFrameDataset(Dataset):
    def __init__(self, ..., *,
                 time_reversal_prob: float = 0.0,
                 time_reversal_perm: torch.Tensor | None = None,
                 time_reversal_allow_mask: torch.Tensor | None = None):
        ...
    def __getitem__(self, index):
        ...
        if self._should_reverse(label):
            indices = list(reversed(indices))
            label = int(self.time_reversal_perm[int(label)].item())
        ...
```

The augmentation is gated by `dataset.time_reversal_prob` (default 0.0). When set to e.g. 0.3, about 30% of clips drawn from reversible classes are reversed and re-labelled at every epoch, which roughly doubles the effective training set for the affected classes without any synthetic data.

5.3 Multi-scale TTA and logit adjustment

The Phase 2 TTA (Section 3.4) already averages horizontal-flip predictions with an auto-derived left/right class permutation. Phase 4 extends inference with two further tricks.

Multi-scale TTA. At inference time we resample the input clip to several spatial scales (e.g. $\{0.875, 1.0, 1.125\} \times H \times W$) using bilinear interpolation, run the model on each scale (optionally with a horizontal flip), softmax each set of logits, and average all softmaxes. This is a standard test-time ensembling trick for CNN classifiers; it is gated by `training.tta_scales` (a list, default [1.0]).

```
def _rescale_video(video_batch, scale):
    if abs(scale - 1.0) < 1e-6:
        return video_batch
    b, t, c, h, w = video_batch.shape
    new_h, new_w = int(round(h * scale)), int(round(w * scale))
    return F.interpolate(video_batch.reshape(b * t, c, h, w),
                         size=(new_h, new_w),
                         mode="bilinear",
                         align_corners=False).view(b, t, c, new_h, new_w)
```

Logit adjustment. Even with a class-balanced sampler and loss, the predictor at test time still inherits a small bias toward head classes whenever the training data and the test data have different (but unknown) class distributions. Menon et al. [10] show that subtracting $\tau \cdot \log \pi_c$ from class c's logit at *inference time*, where π_c is the training-set prior and $\tau \in [0, 1]$ is a small constant, is the Bayes-optimal post-hoc correction under the standard balanced-test assumption. We compute π_c directly from the training-folder counts and subtract $\tau \log \pi_c$ from the logits right after the forward pass, before either the softmax or the multi-scale averaging:

```
def _build_logit_adjustment(train_dir, num_classes, tau, device):
    counts = class_counts(...)
    total = float(sum(counts))
    pi = torch.tensor([c / total if c > 0 else 1e-12 for c in counts],
                      device=device)
    return tau * torch.log(pi) # subtracted from logits

# inside _logits_for_batch:
```

```
logits = model(video_batch)
if logit_adjust is not None:
    logits = logits - logit_adjust
```

The strength τ is configured via `training.tta_logit_adjust` (default 0.0 = off; we use $\tau = 1.0$ in the bundled experiment).

5.4 The bundled experiment configuration

The four Phase-4 features compose cleanly with everything in Phases 1 through 3. A new experiment file `configs/experiment/track_a_vjepa_finetune.yaml` bundles them on top of a V-JEPA warm-start:

```
defaults:
  - override /model: avanced_resnet50_tsm
  - override /augment: randaugment_t

model:
  drop_path_rate: 0.1
  head: attn
  head_num_heads: 4
  init_from: null # set on the command line

dataset:
  num_frames: 4
  use_official_val: true
  time_reversal_prob: 0.3

training:
  optimizer: adamw
  weight_decay: 0.05
  amp: true
  ema_enabled: true
  ema_decay: 0.999
  tta: true
  tta_flip: true
  tta_scales: [0.875, 1.0, 1.125]
  tta_logit_adjust: 1.0
  class_balance_sampler: sqrt_inverse
  class_balance_loss: cb
  class_balance_beta: 0.999
  early_stopping_enabled: true
  early_stopping_patience: 20
```

6 Configuration and reproducibility

6.1 New configuration knobs

The following table summarises every new switch introduced by Phases 1 through 4. All defaults preserve the legacy behaviour.

Key	Default	Effect when on
Phases 1–2		
`dataset.use_official_val`	`false`	validate on `data/val/`
`training.amp`	`false`	FP16 autocast + GradScaler
`training.ema_enabled`	`false`	EMA of model weights
`training.ema_decay`	`0.999`	EMA momentum
`training.tta`	`false`	enable TTA at submit time
`training.tta_flip`	`true`	flip view (only used when `tta`)
`training.tta_temporal_shift`	`0`	extra temporal start-offsets (TTA)
`model.drop_path_rate`	`0.0`	Stochastic Depth maximum drop prob.
`model.head`	`mean`	temporal head (`mean`/`attn`)
`model.head_num_heads`	`4`	MultiHead attention heads
`model.init_from`	`null`	path to SSL trunk to warm-start
Phase 3 (V-JEPA SSL)		
`pretrain.mask_prob`	`0.5`	avg. frame mask fraction for V-JEPA
`pretrain.min_mask`	`1`	min frames masked per clip
`pretrain.max_mask`	`2`	max frames masked per clip
`pretrain.predictor_hidden_dim`	`2048`	MLP predictor hidden width
`pretrain.teacher_momentum_base`	`0.996`	cosine EMA momentum start
`pretrain.teacher_momentum_final`	`1.0`	cosine EMA momentum end
Phase 4		
`training.class_balance_sampler`	`none`	sample weights: `none/inverse/sqrt_inverse`
`training.class_balance_loss`	`none`	class-balanced loss policy (`none/cb`)
`training.class_balance_beta`	`0.999`	β in the CB effective-number formula
`dataset.time_reversal_prob`	`0.0`	prob. of time-reversing a reversible-class clip
`training.tta_scales`	`[1.0]`	list of spatial scales for multi-scale TTA
`training.tta_logit_adjust`	`0.0`	τ in Menon et al. logit adjustment

6.2 New experiment configurations

`configs/experiment/track_a_phase2.yaml`
Bundles every Phase-1 and Phase-2 opt-in: AdamW + WD 0.05, AMP, DropPath 0.1, attention head, EMA 0.999, TTA at submit time, official validation split, cosine schedule with 10-epoch warmup, label smoothing 0.1, RandAugment-T, FrameMixUp.

`configs/experiment/track_a_phase2_balanced.yaml`
Phase 2 *plus* all of Phase 4 (class-balanced sampler + CB loss, time-reversal augmentation, multi-scale TTA, logit adjustment) but *without* SSL warm-start. Useful as an ablation baseline.

`configs/experiment/track_a_vjepa_pretrain.yaml`
Routes the run through `pretrain_vjepa.py`: 50 epochs of V-JEPA-style clip mask-denoising over the union of train+val+test videos, 1–2 frames masked per clip, AdamW + WD 0.04, AMP on.

`configs/experiment/track_a_vjepa_finetune.yaml`
Same as `track_a_phase2_balanced.yaml` but expects `model.init_from` to be set on the command line to the V-JEPA trunk produced by the previous experiment; reduces the number of supervised epochs (warm-started runs converge faster) and halves the learning rate.

`configs/experiment/track_a_ssl_pretrain.yaml, ..._ssl_finetune.yaml`
The legacy DINO-on-still-frames experiments are retained for reference. They are superseded by the V-JEPA variants above and are not recommended for new runs (see Section 4.1).

6.3 Backwards compatibility

Every legacy entry point continues to work without modification:

- Phase-1 checkpoints (no `trained_class_indices`, no `optimizer_state_dict` in `extra`) load and resume via the legacy fast-forward fallback.
- Pre-Phase-1 checkpoints (no Phase-2 opt-ins enabled) load with identical state-dict keys; this is enforced by a unit test that compares state-dict key sets between defaults and Phase-2 opt-ins.
- Submission CSVs produced before Phase 1 remain valid; the new mask is only applied if the checkpoint actually carries the trained-class list.

7 Test coverage

The pre-existing suite contained 120 tests, all of which still pass. Forty-one new tests were added across the four phases, bringing the total to **161 passing tests**:

`tests/shared/models/test_phase2_optins.py` (8 tests)
Phase 2: `DropPath` eval-time identity and unbiased per-sample training behaviour; `DropPath(0)` byte-for-byte equality with the no-op; attention pool shape; state-dict key invariance under `drop_path_rate`; `attn_pool.*` key appearance.

`tests/pipelines/test_submit_tta.py` (5 tests)
Phase 2: auto-derived left/right class permutation; no-TTA path; untrained-mask path; flip-TTA softmax reinforcement of the correct class after remap.

`tests/shared/data/test_still_frames_dataset.py` (6 tests)
Phase 3 (DINO, retained): frame-path discovery across both folder layouts; deduplication across roots; multi-view sampling shape.

`tests/shared/models/test_dino_ssl.py` (4 tests)
Phase 3 (DINO, retained): SSL trunk key superset of the supervised backbone keys; finite back-propagating loss; teacher-EMA midpoint identity.

`tests/pipelines/test_init_from_ssl.py` (1 test)
Phase 3 (DINO, retained) end-to-end smoke test for the `model.init_from` hook.

`tests/shared/models/test_vjepa_ssl.py` (10 tests)
Phase 3 (V-JEPA): trunk forward shape; remapped V-JEPA trunk keys cover the supervised backbone *exactly* (zero missing, zero unexpected); `load_state_dict` only misses the classifier; the L1 loss is zero for an empty mask (with a valid `grad_fn`) and finite-and-backprop-able otherwise; `apply_frame_mask` zeroes the correct positions; mask sampler bounds and "never-all-masked" guarantee; teacher EMA midpoint.

`tests/shared/data/test_clip_ssl_dataset.py` (7 tests)
Phase 3 (V-JEPA): both folder layouts; cross-root deduplication; missing-root skip; sample shape; temporal jitter under hold-out.

`tests/pipelines/test_init_from_vjepa.py` (1 test)
Phase 3 (V-JEPA) end-to-end smoke test that explicitly pins the no-double-`backbone.`-prefix invariant of the V-JEPA save format.

`tests/shared/utils/test_class_balance.py`,
`tests/shared/data/test_time_reversal.py`,
`tests/pipelines/test_submit_phase4_tta.py`
Phase 4: per-class weight policies (`none`, `inverse`, `sqrt_inverse`, `cb`) and their corner cases (zero-count class set to weight 0); time-reversal table-building and label-remap symmetry (pair-permuted twice = identity); multi-scale TTA shape and logit-adjustment numerics.

The full suite is exercised by:

```
PYTHONPATH=src .venv/bin/python -m pytest
```

8 Expected impact and launch strategy

8.1 Per-change order-of-magnitude estimates

Based on the literature and on the gap currently observed between internal validation and the public leaderboard, we expect the following contributions to Track A's public Top-1 score (these are order-of-magnitude estimates, not promises):

Change	Internal val	Public Top-1
Baseline (current best)	0.5666	0.3584
+ official val split (Phase 1)	lower *	–
+ Kaiming init (Phase 1)	+0.5–2 pt	+0.3–1 pt
+ AMP (Phase 1)	speedup only	speedup only
+ optimizer/scheduler state on resume (Phase 1)	0–1 pt	0–1 pt
+ untrained-class masking (Phase 1)	negligible	+0.5–1.5 pt
+ DropPath (Phase 2)	+0.5–1 pt	+0.3–0.8 pt
+ Attention head (Phase 2)	+0.5–1 pt	+0.3–0.8 pt
+ EMA model averaging (Phase 2)	+0.3–0.8 pt	+0.2–0.6 pt
+ TTA flip with class-pair remap (Phase 2)	+0–0.5 pt	+0.5–1 pt
+ AdamW + WD 0.05 (Phase 2)	+0.3–1 pt	+0.3–1 pt
+ DINO SSL pretraining (deprecated)	–1 to +1 pt †	–1 to +1 pt †
+ V-JEPA SSL pretraining (Phase 3)	+2–4 pt	+2–4 pt
+ Class-balanced sampler + CB loss (Phase 4)	+1–2 pt	+1–2 pt
+ Time-reversal augmentation (Phase 4)	+0.5–1.5 pt	+0.5–1.5 pt
+ Multi-scale TTA (Phase 4)	+0.3–0.8 pt	+0.3–0.8 pt
+ Logit adjustment (Phase 4)	+0.2–0.5 pt	+0.2–0.5 pt

* The official split is harder than the leaky internal split, so reported val will drop — but the public score will move closer to the val score.

† As observed in practice and explained in Section 4.1: DINO on still frames produces inconsistent downstream gains. We retain the row in this table only as a baseline against which V-JEPA's stable gains stand out.

8.2 Recommended launch order

We recommend running the V-JEPA pretrain → balanced fine-tune → multi-scale TTA submit sequence end-to-end. Approx. runtime on a single Ampere GPU at batch 16: ∼20–25 h total with AMP. The $LOG variable below is expected to point at a unique log file under `logs/`.

Stage 1 — V-JEPA pretrain. Trains the TSM-equipped ResNet-50 trunk by frame mask-denoising over train+val+test clips (**no** labels are consumed, so this is Track-A-compliant).

```
nohup env PYTHONUNBUFFERED=1 PYTHONPATH=src \
    .venv/bin/python -u -m smth2smth.pipelines.pretrain_vjepa \
    track=a experiment=track_a_vjepa_pretrain \
    pretrain.checkpoint_path=$(pwd)/vjepa_trunk.pt \
    > "$LOG" 2>&1 &
```

Stage 2 — Supervised fine-tune from the V-JEPA trunk. Bundles every Phase-1, Phase-2, and Phase-4 opt-in (class balance, time-reversal, multi-scale TTA, logit adjustment).

```
nohup env PYTHONUNBUFFERED=1 PYTHONPATH=src \
    .venv/bin/python -u -m smth2smth.pipelines.train \
    track=a experiment=track_a_vjepa_finetune \
```

```
    model.init_from=$(pwd)/vjepa_trunk.pt \
    > "$LOG" 2>&1 &
```

Stage 3 — Submission with TTA + logit adjustment. `tta_scales` and `tta_logit_adjust` are already on by default for this experiment, so no extra CLI overrides are needed.

```
nohup env PYTHONUNBUFFERED=1 PYTHONPATH=src \
    .venv/bin/python -u -m smth2smth.pipelines.submit \
    track=a experiment=track_a_vjepa_finetune \
    > "$LOG" 2>&1 &
```

For a single-stage ablation (no V-JEPA pretraining), use `experiment=track_a_phase2_balanced` in Stages 2 and 3 instead.

9 Summary of files touched

`src/smth2smth/pipelines/train.py`
Phase 1 (official val, AMP, optimizer/scheduler/scaler state, trained-class list) + Phase 2 (EMA + best-of-(live, EMA) selection) + Phase 3 (`model.init_from` hook with key remap helper, handles both DINO and V-JEPA trunk key flavours) + Phase 4 (`WeightedRandomSampler`, class-balanced loss weights routed through cross-entropy and the soft-target path, time-reversal table wiring).

`src/smth2smth/pipelines/submit.py`
Phase 1 (untrained-class masking) + Phase 2 (flip-TTA with auto-derived left/right class permutation) + Phase 4 (multi-scale TTA via `_rescale_video`, post-hoc logit adjustment via `_build_logit_adjustment`).

`src/smth2smth/pipelines/pretrain_vjepa.py` **(new, Phase 3)**
V-JEPA-style clip mask-denoising pretraining loop with EMA teacher and trunk-only checkpoint save.

`src/smth2smth/pipelines/pretrain_ssl.py` **(retained, deprecated)**
Legacy DINO-v1 pretraining loop on still frames; kept for reference.

`src/smth2smth/shared/engine/trainer.py`
Phase 1 AMP wiring + Phase 2 `ema_model` hook + Phase 4 class-weight propagation through `_soft_target_cross_entropy`.

`src/smth2smth/shared/models/avanced_resnet50_tsm.py`
Phase 1 Kaiming init + Phase 2 `DropPath`, `AttentionPool`, patched `Bottleneck.forward`.

`src/smth2smth/shared/models/vjepa_ssl.py` **(new, Phase 3)**
`VJepaTrunk` (TSM-wrapped ResNet-50), `VJepaPredictor`, `VJepaModel`, `make_frame_mask`/`apply_frame_mask`, `vjepa_feature_loss`, `update_vjepa_teacher_ema`.

`src/smth2smth/shared/models/dino_ssl.py` **(retained, deprecated)**
Phase 3 (legacy) `DinoModel`, `DinoHead`, `DinoLoss`.

`src/smth2smth/shared/data/clip_ssl_dataset.py` **(new, Phase 3)**
`collect_all_video_dirs` (handles class-bucketed and flat layouts) + `ClipSSLDataset` (label-free clip iterator).

`src/smth2smth/shared/data/video_dataset.py`
Phase 4: `time_reversal_prob`, `time_reversal_perm`, `time_reversal_allow_mask` keyword arguments threaded into `VideoFrameDataset.__getitem__`.

`src/smth2smth/shared/data/time_reversal.py` **(new, Phase 4)**
Hand-curated reversible class-pair table and `build_time_reversal_table`/`describe_table` helpers.

`src/smth2smth/shared/utils/class_balance.py` **(new, Phase 4)**
`class_counts`, `compute_sample_weights`, `compute_class_weights` for sampler and loss weighting.

`configs/data/default.yaml`
Phase 1 `use_official_val`; Phase 4 `time_reversal_prob`.

`configs/train/default.yaml`
Phase 1 `amp`, Phase 2 `ema_*`, `tta_*`; Phase 4 `class_balance_*`, `tta_scales`, `tta_logit_adjust`.

`configs/model/avanced_resnet50_tsm.yaml`
Phase 2 `drop_path_rate`, `head`, `head_num_heads`; Phase 3 `init_from`.

`configs/pretrain/default.yaml`
Phase 3 SSL (DINO) pretraining defaults (retained).

`configs/pretrain/vjepa.yaml` **(new, Phase 3)**
V-JEPA pretraining defaults.

`configs/experiment/track_a_phase2.yaml`
Phase 2 bundle.

`configs/experiment/track_a_phase2_balanced.yaml` **(new, Phase 4)**
Phase 2 + every Phase-4 opt-in (no SSL).

`configs/experiment/track_a_vjepa_pretrain.yaml` **(new, Phase 3)**
V-JEPA pretrain bundle.

`configs/experiment/track_a_vjepa_finetune.yaml` **(new, Phase 3+4)**
V-JEPA warm-start + every Phase-4 opt-in.

`configs/experiment/track_a_ssl_pretrain.yaml`,
`configs/experiment/track_a_ssl_finetune.yaml`
Legacy DINO bundles, retained for reference; superseded.

`configs/config.yaml`
`pretrain: default` in the defaults list.

`tests/ ...`
41 new unit and integration tests (120 → 161 passing).

10 Conclusion

Phase 1 fixes the silent correctness issues that were inflating validation scores and silently degrading test performance. Phase 2 layers on four well-understood, opt-in techniques — Stochastic Depth, an attention-pool head, EMA model averaging, and flip-TTA with auto-derived class-pair remap — that compose cleanly with each other and with the existing RandAugment-T + FrameMixUp data pipeline. Phase 3 replaces an earlier DINO-on-still-frames SSL prototype — which was structurally incompatible with the TSM trunk because it left the channel-shift modules unexercised — by a V-JEPA-style clip-level mask-denoising pretraining that operates on full T-frame clips and exercises the TSM channels by construction; the resulting trunk loads back into the supervised model via the same `model.init_from` hook with zero missing backbone keys. Phase 4 fixes the remaining small-dataset failure modes with a class-balanced sampler, an effective-number class-balanced loss, a motion-aware time-reversal augmentation that pairs reversible verb classes, multi-scale TTA,

and a post-hoc logit adjustment. All changes default to off, the test suite has grown from 120 to 161 passing tests, and the existing experiment configurations and resume paths continue to work unchanged.

References

[1] K. He, X. Zhang, S. Ren, J. Sun. *Delving Deep into Rectifiers: Surpassing Human-Level Performance on ImageNet Classification.* ICCV 2015.

[2] G. Huang, Y. Sun, Z. Liu, D. Sedra, K. Q. Weinberger. *Deep Networks with Stochastic Depth.* ECCV 2016.

[3] M. Caron, H. Touvron, I. Misra, H. Jégou, J. Mairal, P. Bojanowski, A. Joulin. *Emerging Properties in Self-Supervised Vision Transformers.* ICCV 2021.

[4] B. T. Polyak, A. B. Juditsky. *Acceleration of Stochastic Approximation by Averaging.* SIAM J. Control and Optimization, 1992.

[5] J. Lin, C. Gan, S. Han. *TSM: Temporal Shift Module for Efficient Video Understanding.* ICCV 2019.

[6] I. Loshchilov, F. Hutter. *Decoupled Weight Decay Regularization.* ICLR 2019.

[7] A. Bardes, Q. Garrido, J. Ponce, X. Chen, M. Rabbat, Y. LeCun, M. Assran, N. Ballas. *Revisiting Feature Prediction for Learning Visual Representations from Video* (V-JEPA). TMLR 2024.

[8] M. Assran et al. *V-JEPA 2: Scaling Joint-Embedding Predictive Architectures for Self-Supervised Video Representation Learning.* arXiv:2506.09985, 2025.

[9] Y. Cui, M. Jia, T.-Y. Lin, Y. Song, S. Belongie. *Class-Balanced Loss Based on Effective Number of Samples.* CVPR 2019.

[10] A. K. Menon, S. Jayasumana, A. S. Rawat, H. Jain, A. Veit, S. Kumar. *Long-Tail Learning via Logit Adjustment.* ICLR 2021.